CSE221: Algorithms

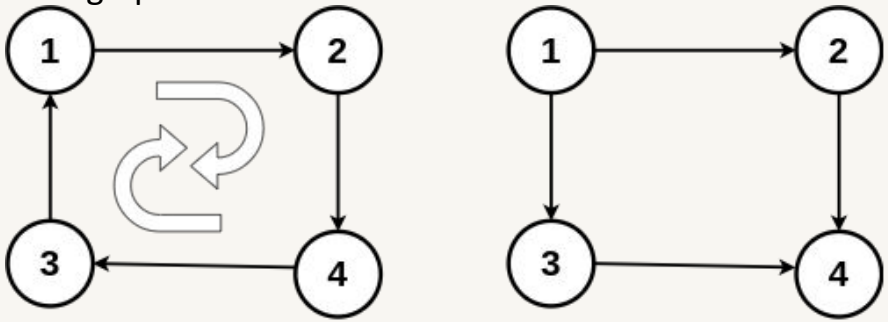
Topological Sorting

Prepared by: A B M Muntasir Rahman [MUNR]



Directed Acyclic Graph (DAG)

- A **Directed Acyclic Graph (DAG)** is a directed graph that does not contain any cycles.
- Directed Acyclic Graph has two important features:
  - **Directed Edges:** In Directed Acyclic Graph, each edge has a direction, meaning it goes from one vertex (node) to another.
  - **Acyclic:** The term "acyclic" indicates that there are no cycles or closed loops within the graph.

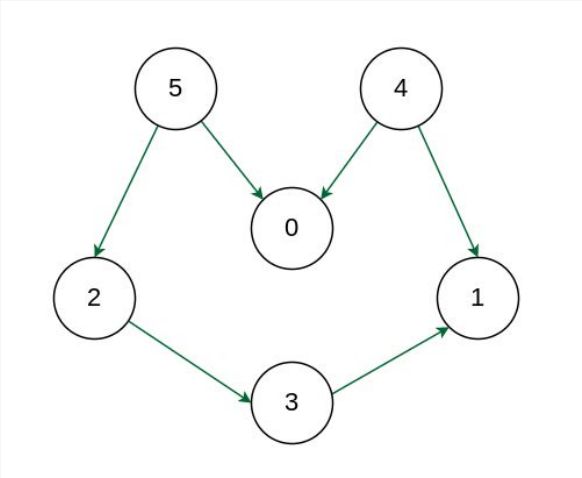


Direct Cyclic Graph

Direct Acyclic Graph

Topological Sorting

- **A linear ordering of vertices such that for every directed edge (u, v), vertex u comes before v in the ordering.**
- Only works for Directed Acyclic Graph (DAG).



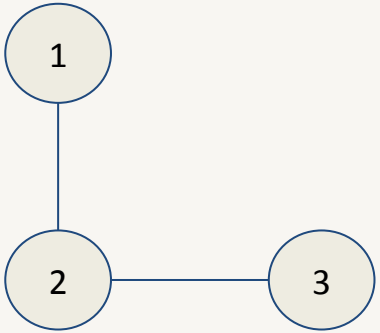
Here, **E = {(2, 3), (3, 1), (4, 0), (4, 1), (5, 0), (5, 2)}**

One possible topological sorting is 4 5 2 3 1 0. Among the edges, one of them is (4, 0). As we can see, 4 comes before 0 in the sequence. Same can be said for (3, 1) and others as well.

There can be more than one topological sorting for a graph.

Topological Sorting

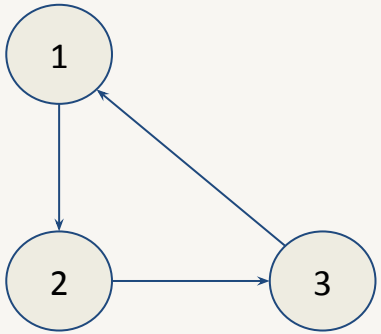
- Why topological sorting is not possible for graphs with undirected edges?
  - This creates a contradiction in the topological ordering as an undirected edge (u, v) means there is an edge from u to v as well as from v to u.



Here, the edge between nodes 1 and 2 means just they are connected, but there's no concept of one coming before the other. As such, contradiction will be created in the ordering.

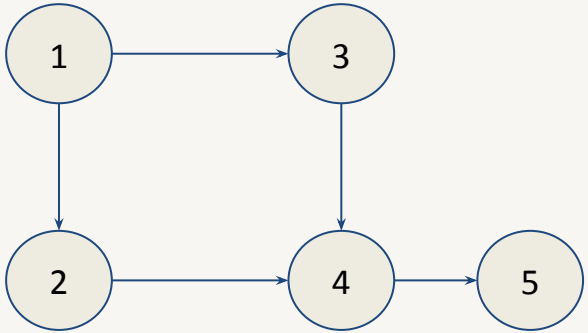
Topological Sorting

- Why topological sorting is not possible for graphs with cycles?
  - This also creates a contradiction in the topological ordering as in a cycle, all the vertices are indirectly dependent on each other.



Here, the vertices (1, 2), (2, 3) and (3, 1) are indirectly dependent on each other. As a result, a topological ordering 1 2 3 contradicts with the edge (3, 1) as here 3 comes before 1.

Topological Sorting using DFS



Visited

0	0	0	0	0
1	2	3	4	5

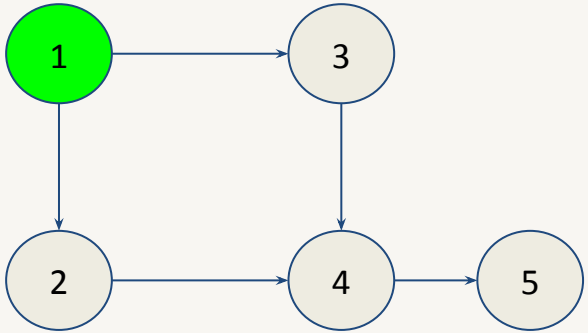
Start Time

0	0	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	0	0	0	0
1	2	3	4	5

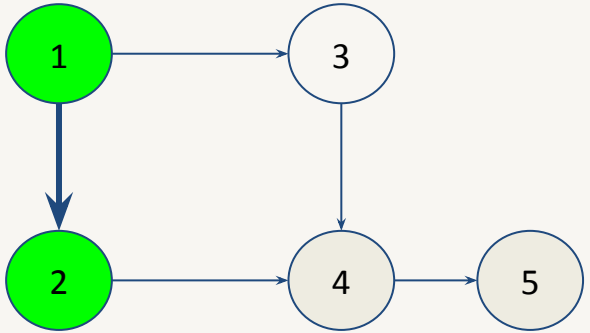
Start Time

1	0	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	0	0
1	2	3	4	5

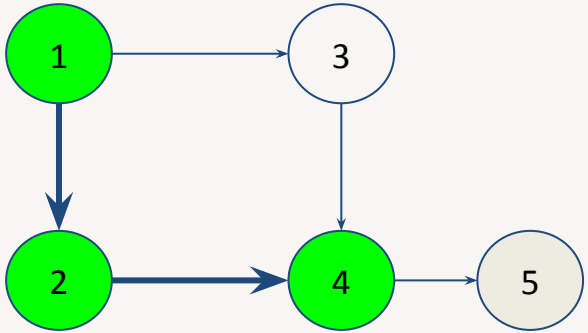
Start Time

1	2	0	0	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	0
1	2	3	4	5

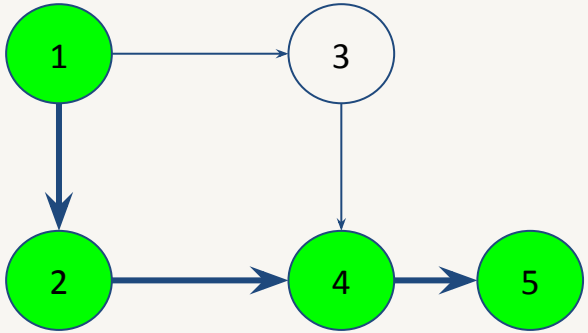
Start Time

1	2	0	3	0
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	1
1	2	3	4	5

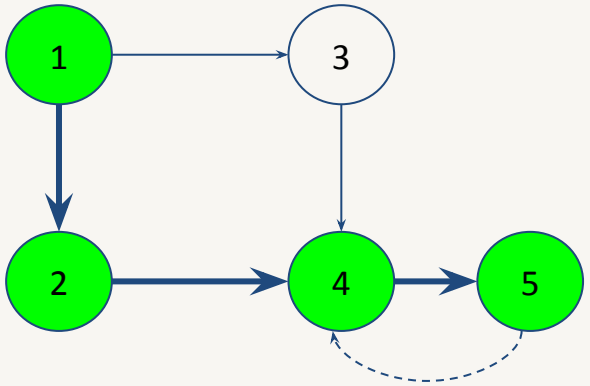
Start Time

1	2	0	3	4
1	2	3	4	5

End Time

0	0	0	0	0
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	1
1	2	3	4	5

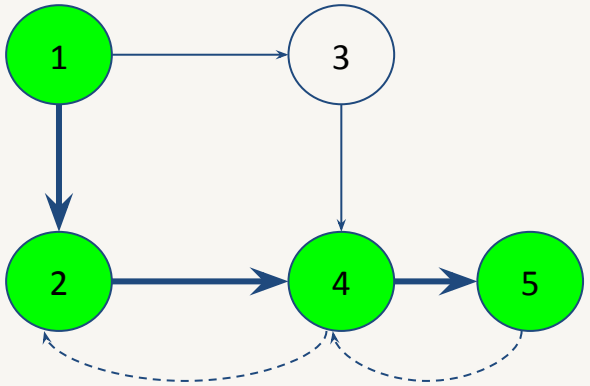
Start Time

1	2	0	3	4
1	2	3	4	5

End Time

0	0	0	0	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	1
1	2	3	4	5

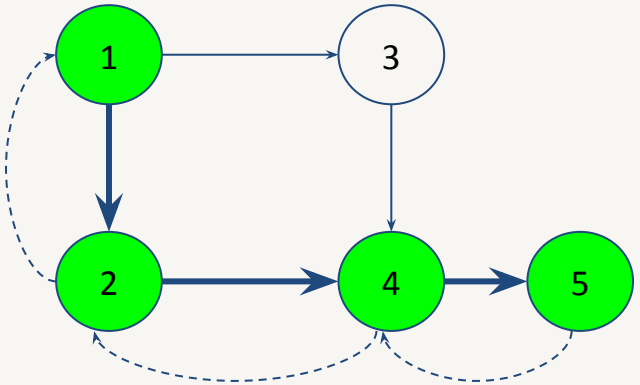
Start Time

1	2	0	3	4
1	2	3	4	5

End Time

0	0	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	0	1	1
1	2	3	4	5

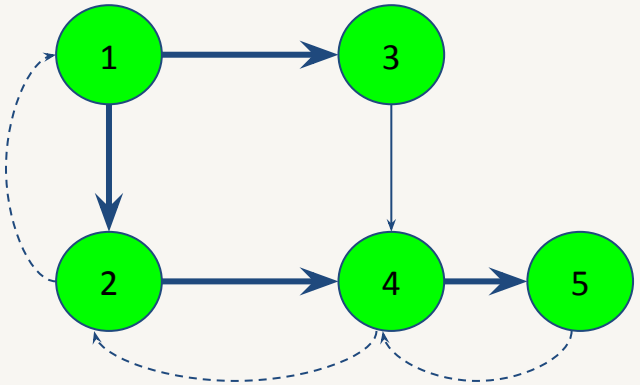
Start Time

1	2	0	3	4
1	2	3	4	5

End Time

0	7	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	1	1	1
1	2	3	4	5

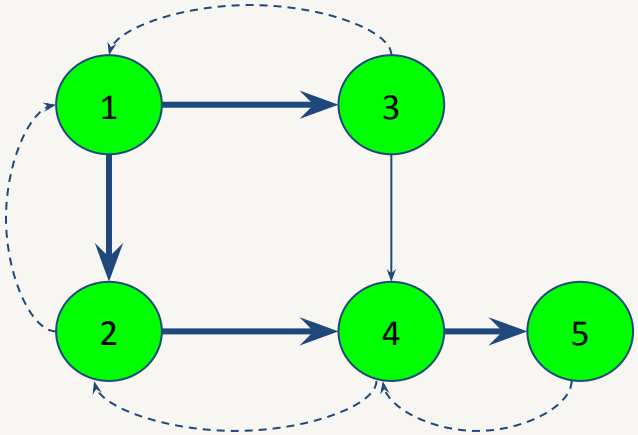
Start Time

1	2	8	3	4
1	2	3	4	5

End Time

0	7	0	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	1	1	1
1	2	3	4	5

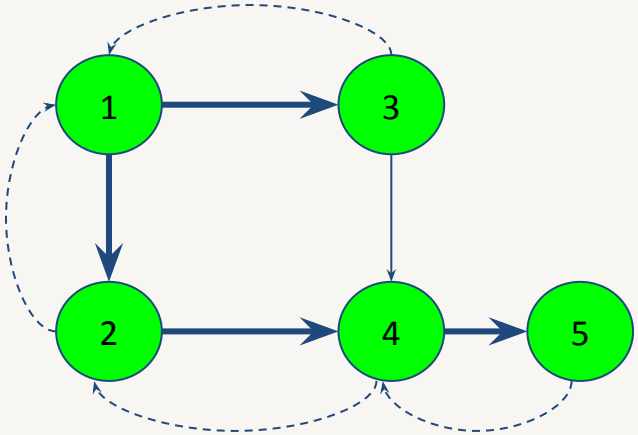
Start Time

1	2	8	3	4
1	2	3	4	5

End Time

0	7	9	6	5
1	2	3	4	5

Topological Sorting using DFS



Visited

1	1	1	1	1
1	2	3	4	5

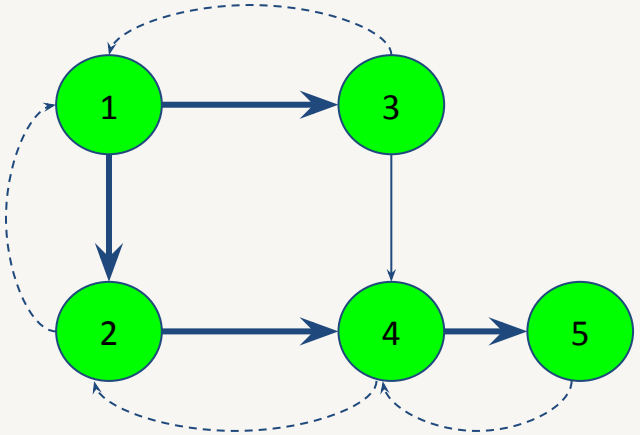
Start Time

1	2	8	3	4
1	2	3	4	5

End Time

10	7	9	6	5
1	2	3	4	5

Topological Sorting using DFS



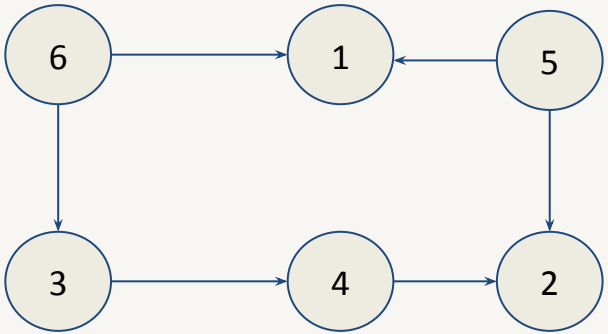
End Time

10	7	9	6	5
1	2	3	4	5

If we sort the ending times based on descending order, we get 10, 9, 7, 6, 5 i.e. vertices 1, 3, 2, 4, 5 which is a topological ordering for the above graph.

Topological Sorting using BFS

- Also known as Kahn's Algorithm for Topological Sorting.



Indegree

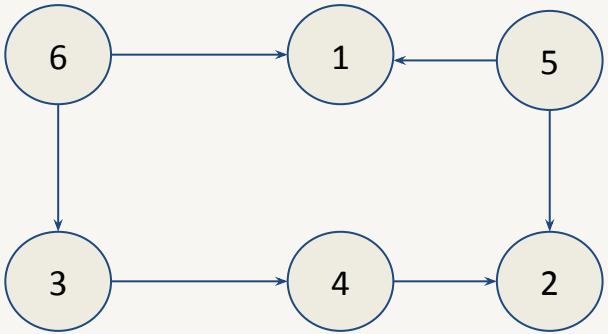
2	2	1	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering:

Topological Sorting using BFS

- Initially, enqueue all the nodes with indegree 0 into the queue i.e. 5 and 6 in this case.



Indegree

2	2	1	1	0	0
1	2	3	4	5	6

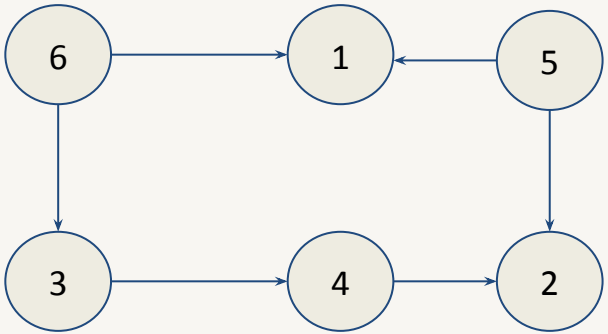
5 6

Queue

Topological Ordering:

Topological Sorting using BFS

- Dequeue from the front of the queue i.e. 5 in this case.



Indegree

2	2	1	1	0	0
1	2	3	4	5	6

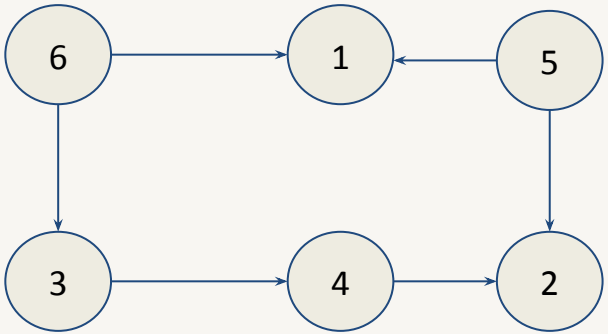
6

Queue

Topological Ordering:

Topological Sorting using BFS

- Now, reduce the indegree of the adjacent vertices of 5 i.e. 1 and 2.



Indegree

1	1	1	1	0	0
1	2	3	4	5	6

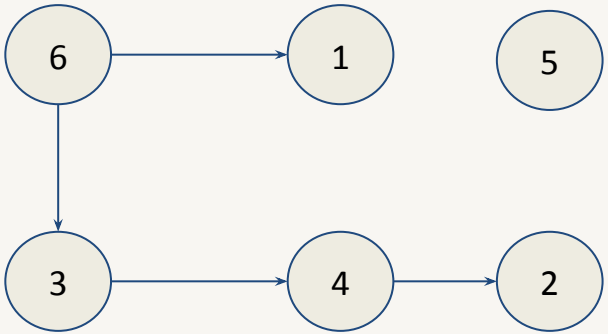
6

Queue

Topological Ordering: 5

Topological Sorting using BFS

- Next, remove from the front of the queue i.e. 6 in this case.



Indegree

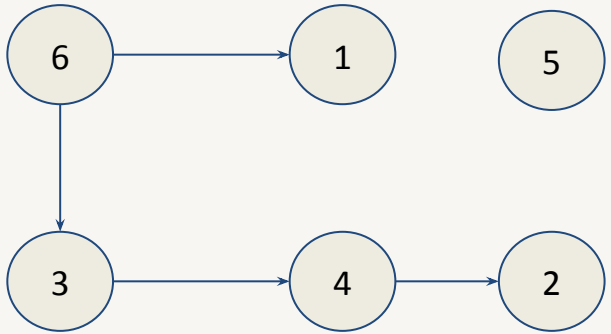
1	1	1	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5

Topological Sorting using BFS

- Now, reduce the indegree of the adjacent vertices of 6 i.e. 1 and 3.



Indegree

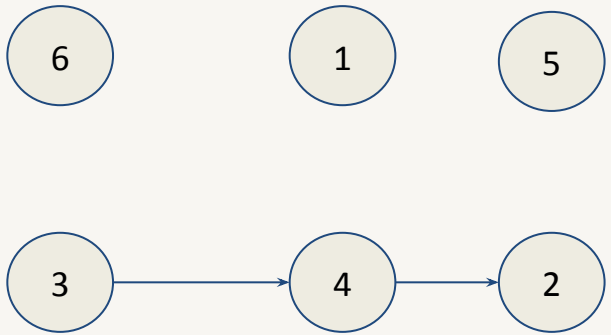
0	1	0	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6

Topological Sorting using BFS

- Indegree of vertices 1 and 3 is 0 now. Therefore, insert them into the queue.



Indegree

0	1	0	1	0	0
1	2	3	4	5	6

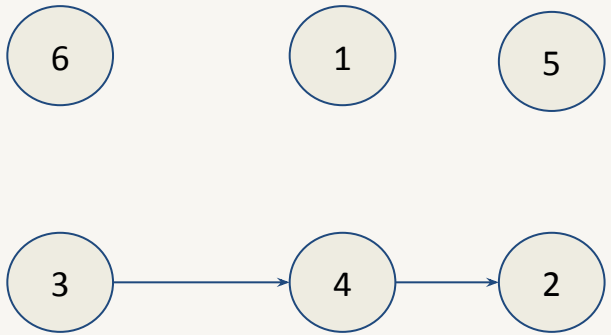
1 3

Queue

Topological Ordering: 5 6

Topological Sorting using BFS

- Remove from the front of queue i.e. 1. Now, 1 does not have any outgoing edges so go to next step and write 1 in the ordering.



Indegree

0	1	0	1	0	0
1	2	3	4	5	6

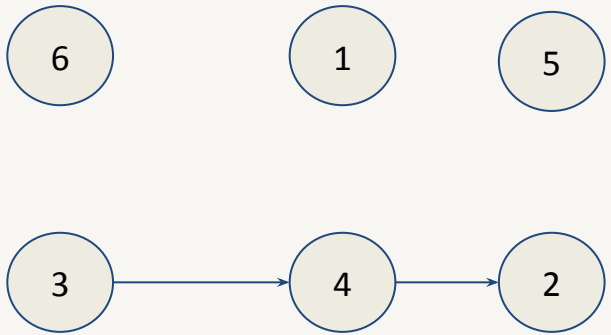
3

Queue

Topological Ordering: 5 6 1

Topological Sorting using BFS

- Again, remove from the front of queue i.e. 3.



Indegree

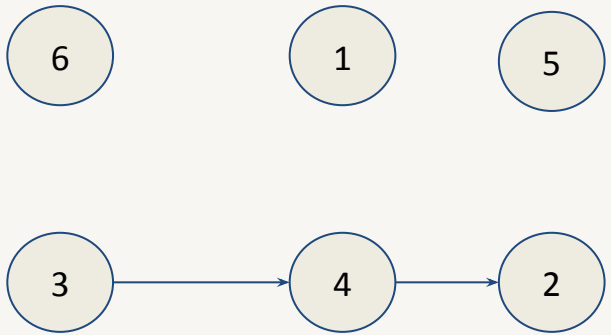
0	1	0	1	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1

Topological Sorting using BFS

- Reduce the indegree of adjacent vertices of 3 i.e. 4 in this case.



Indegree

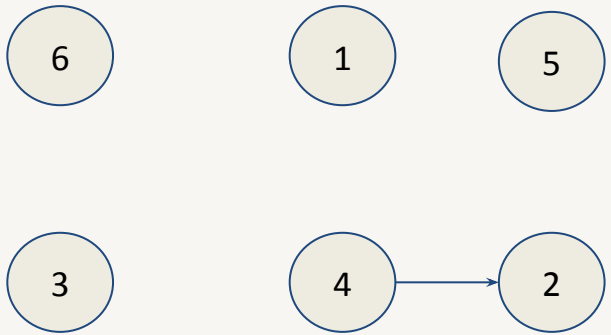
0	1	0	0	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1 3

Topological Sorting using BFS

- Indegree of vertex 4 is 0 now. Enqueue it into the queue.



Indegree

0	1	0	0	0	0
1	2	3	4	5	6

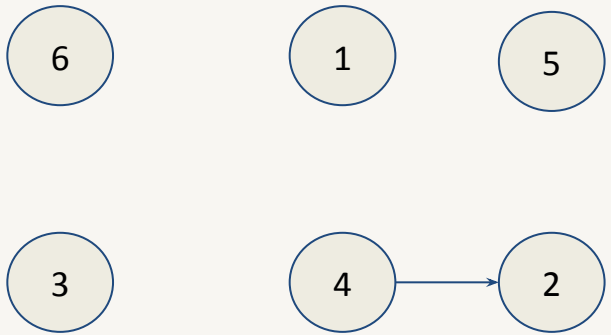
4

Queue

Topological Ordering: 5 6 1 3

Topological Sorting using BFS

- Dequeue from the queue i.e. 4. Reduce the indegree of adjacent vertices of 4 i.e. 2.



Indegree

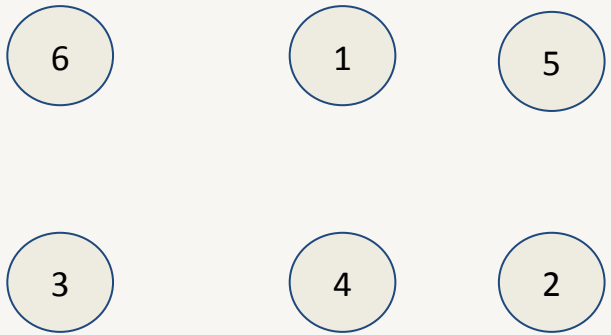
0	0	0	0	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1 3 4

Topological Sorting using BFS

- Enqueue 2 into the queue has it has indegree 0.



Indegree

0	0	0	0	0	0
1	2	3	4	5	6

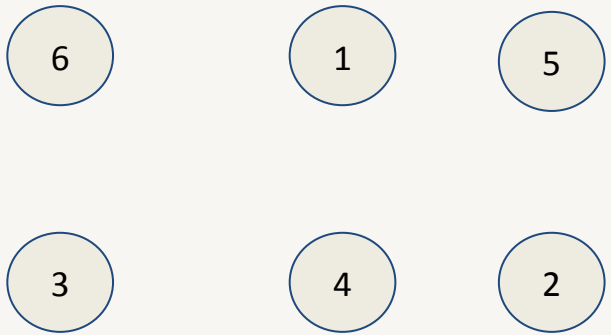
2

Queue

Topological Ordering: 5 6 1 3 4

Topological Sorting using BFS

- Dequeue from the front of queue, and since 2 has no adjacent vertices, write it in the ordering. 5 6 1 3 4 2 is topological sorting for the graph.



Indegree

0	0	0	0	0	0
1	2	3	4	5	6

Queue

Topological Ordering: 5 6 1 3 4 2